

SOFTWARE PROJECT PLAN

Wildlife Information System

(WIS)

Document Number: WIS-PP-001

Version: 2.0

Date: September 2026

Prepared by

Muhammad Abbas
Muhammad Ehtisham
Fahad Naeem Khan

Front Matter

Document Control

Document title	Software Project Plan — Wildlife Information System (WIS)
Document number	WIS-PP-001
Version	2.0
Date	September 2026
Status	Draft for team and supervisor review
Prepared by	Muhammad Abbas, Muhammad Ehtisham, Fahad Naeem Khan
Source document	Wildlife Information System SRS, Version 2.0, June 2026 (WIS-SRS-001)

Revision History

Version	Date	Description
1.0	June 2026	Initial requirements captured in the SRS (WIS-SRS-001).
2.0	September 2026	Project Plan prepared from SRS Version 2.0, organised into overview, references, definitions, context, planning, supporting process plans and risk management.

Purpose of This Document

This Project Plan describes how the Wildlife Information System will be planned and organized, built, tested and delivered. It reorganizes the requirements, scope, technology choices, schedule and test strategy defined in the WIS SRS Version 2.0 into a project-management view. The risk-management section is a planning addition derived from the assumptions, dependencies, scope and architecture in the SRS; it should be reviewed and updated by the team during development.

Contents

1. Project Overview
2. References
3. Definitions
4. Project Context
5. Project Planning
6. Supporting Process Plans
7. Risk Management
8. Key System Interface and API Requirements
9. Implementation Status
10. Traceability to the Plan Outline

1. Project Overview

1.1 Project Summary

The Wildlife Information System (WIS) is a centralized system for storing and sharing wildlife information in one organized database. In Pakistan and many other countries, wildlife data is scattered across government departments, universities, conservation organizations, spreadsheets, reports, notebooks, paper records and private GPS/IoT systems. WIS brings this data together and makes it available to authorized users.

The system covers animal profiles, population information, movement records, DNA and biological data, health and veterinary records, and behavior and environment data. It provides a RESTful API with JSON responses for authorized access and a basic web interface for searching and manual data entry.

1.2 Purpose, Scope and Objectives

Purpose: Reduce wildlife-data fragmentation and provide a central repository that supports research, wildlife monitoring, conservation projects and decision making.

Scope: A central PostgreSQL database; a backend that validates, organizes and manages incoming data (field researchers, IoT tracking devices, lab results); a RESTful API; role-based access for Data Entry Operators, Researchers, Government Officials, System Administrators and API developers; and a basic web interface for search and manual data entry.

Primary objective: Build a centralized wildlife information database that is accurate, well organized and accessible through an API for research and analysis.

Secondary objectives:

- Encourage data sharing among wildlife organizations.
- Provide a platform that supports conservation and research decision making.
- Build a scalable foundation that can grow with more data and users over time.

1.3 Assumptions and Constraints

- Users of the web portal and API have constant network connectivity.
- WIS acts strictly as a storage repository for GPS data generated by external tracking hardware; it does not manufacture or install tracking devices.
- Version 1.0 operates in English only.
- External applications access WIS programmatically through the RESTful API using JWT tokens.
- The project is delivered by a three-member team on a fixed ten-week schedule.

1.4 Project Deliverables

#	Deliverable	Phase
D1	Requirements and SRS documentation	Phase 1
D2	PostgreSQL database design / schema (ER diagram)	Phase 2

#	Deliverable	Phase
D3	RESTful API design and endpoint specification	Phase 2
D4	Authentication and role-management design (JWT, bcrypt)	Phase 2
D5	Basic HTML/CSS/JavaScript web interface design	Phase 2
D6	Node.js/Express backend and database setup	Phase 3
D7	Working RESTful API with test results	Phase 4
D8	Testing plan and test cases	Phases 4–5
D9	Final project documentation and review	Phase 5

1.5 Schedule Summary

Phase	Activity	Duration
Phase 1	Requirements gathering and analysis	2 weeks
Phase 2	System design (DB schema, API design)	2 weeks
Phase 3	Backend development and database setup	3 weeks
Phase 4	API development and testing	2 weeks
Phase 5	Documentation and final review	1 week
Total		10 weeks

1.6 User Roles

Role	Access	Capabilities
Admin	Full	Supervise user accounts, perform CRUD on all entities, and modify configurations.
Data Entry Operator	Write + Read	Create or modify wildlife profiles, including tracking, medical and genetic data.
Researcher	Read + Export	Query data through the portal/API and download records in CSV or JSON.
Government Official	Read Only	View and search existing animal information and profiles.
Developer / API Client	API Read	Retrieve data from authenticated public API endpoints using JWT tokens.

2. References

- Wildlife Information System SRS, Version 2.0, June 2026 (WIS-SRS-001).
- Pressman, R. S., Software Engineering: A Practitioner's Approach.
- Mohapatra, P. K. J., Software Engineering: A Lifecycle Approach.

3. Definitions

Term	Definition
WIS	Wildlife Information System
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
FR	Functional Requirement
NFR	Non-Functional Requirement
UC	Use Case
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
JSON	JavaScript Object Notation
DFD	Data Flow Diagram
ER	Entity-Relationship
SRS	Software Requirements Specification

4. Project Context

4.1 Existing System and Problem

Area	Description
Existing system	Government forest departments keep paper or spreadsheet records; universities store field data in local, non-public databases; international bodies (WWF, IUCN) maintain global databases that lack local detail; GPS-collar data is kept private.
Main issues	Data isolation, no standard format, lack of open access, missed analytical opportunities (e.g. disease outbreaks, migration changes, population decline) and duplicated research effort.
Problem statement	No single system in Pakistan stores and shares wildlife information, so researchers duplicate data collection and agencies may decide without complete data.
Proposed solution	A centralized wildlife database with controlled web access and a secure RESTful API.
Primary entities	Animal (central), Species, Movement Record, DNA Sample, Health Record, User and Location. One Animal has many Movement, Health and DNA records; each Animal belongs to one Species.

4.2 Technological Overview

Component	Technology
Backend	Node.js with Express.js
Database	PostgreSQL 14+ (relational, structured data)

Component	Technology
API	RESTful API with JSON responses
Authentication	JWT (JSON Web Tokens); passwords hashed with bcrypt and salt
Frontend	HTML, CSS, JavaScript (basic web interface)
Version control	Git / GitHub
API testing	Postman
Editor / browsers	VS Code; Chrome / Firefox
Hosting (future)	Cloud (AWS or DigitalOcean)
Data formats	JSON; CSV for exports and bulk upload

4.3 System Architecture

WIS follows a 3-tier architecture. External users and applications communicate with the application layer through the RESTful API.

Layer	Component	Responsibility
Presentation	HTML/CSS/JS web interface	Basic UI for data entry, search and export
Application	Node.js + Express.js API server	Business logic, authentication, query processing
Data	PostgreSQL database	Stores wildlife records, users and audit logs

4.4 Functional Requirements

ID	Requirement	Description
FR-01	Animal Profile Management	Add, update and remove animal records; system-generated unique IDs; validation of submitted data (species, age, weight, etc.).
FR-02	Movement Data Recording	Store GPS coordinates and timestamps linked to animal IDs; CSV batch upload; coordinate validation before saving.
FR-03	DNA & Biological Data	Store DNA samples, lab results and species confirmation, linked to an animal profile.
FR-04	Health & Veterinary Records	Record vaccinations, diagnosed conditions and treatments, with date of service and a brief summary.
FR-05	RESTful API Access	Authenticated JSON endpoints for resources such as animals, movements and species; token required on every request.
FR-06	User Management	Admins create/update/deactivate users and assign roles; bcrypt password hashing; unique, verified email.
FR-07	Search & Filter	Search by species, location and time frame with paginated results.
FR-08	CSV Bulk Upload	Validate every row, save valid rows and report skipped rows with reasons.
FR-09	Data Export	Export search results as CSV or JSON via web interface and API.

4.5 Non-Functional Requirements

ID	Type	Requirement
NFR-01	Performance	Standard API queries return within 2 seconds.
NFR-02	Scalability	Database handles up to 1 million animal records without performance loss.
NFR-03	Security	All API calls require JWT; passwords hashed with bcrypt and salted.
NFR-04	Availability	99% availability after deployment.
NFR-05	Data Integrity	All records carry timestamps and audit logs showing who created or modified them.
NFR-06	Usability	Simple web interface usable by non-technical data entry operators without training.
NFR-07	Compatibility	REST API works with any language or tool able to consume REST/JSON.

4.6 Use-Case Summary

ID	Use Case	Actor	Purpose
UC-01	Add Animal Record	Data Entry Operator	Enter validated animal details and save a record with a unique ID.
UC-02	Query Animal Data via API	Researcher / External App	Retrieve matching animal records as JSON using a valid JWT.
UC-03	Upload CSV Movement Data	Data Entry Operator	Upload and validate movement data (animal_id, lat, lng, timestamp).
UC-04	Search Animals	Researcher / Gov. Official	Search by species/location with optional date range (20 per page).
UC-05	Manage Users	System Administrator	Create users and update roles/status.
UC-06	View Health Records	Researcher / Veterinarian	Retrieve health records for a selected animal.
UC-07	Export Data	Researcher	Export filtered records as CSV or JSON.
UC-08	User Login / Authentication	All Users	Authenticate and receive a JWT token.
UC-09	Add DNA Sample Record	Data Entry Operator	Create a DNA record linked to an existing animal.

4.7 Scope Exclusions

- No installation, manufacturing or design of GPS collars or tracking devices.
- No real-time live tracking maps or animated movement visualization (possible future extension).
- No Android/iOS mobile application; third parties may build one on the public API.
- No AI/ML-based animal identification or pattern recognition.
- No direct integration with international databases (e.g. IUCN Red List, CITES) in v1.0.

- No financial, HR, payroll or administrative management features.
- No offline field data-collection tools.
- No automatic alerts for disease outbreaks, environmental risks or illegal hunting.
- No multilingual support in v1.0 (English only).
- No wildlife permit, legal reporting or enforcement tracking.

5. Project Planning

5.1 Development Process Model

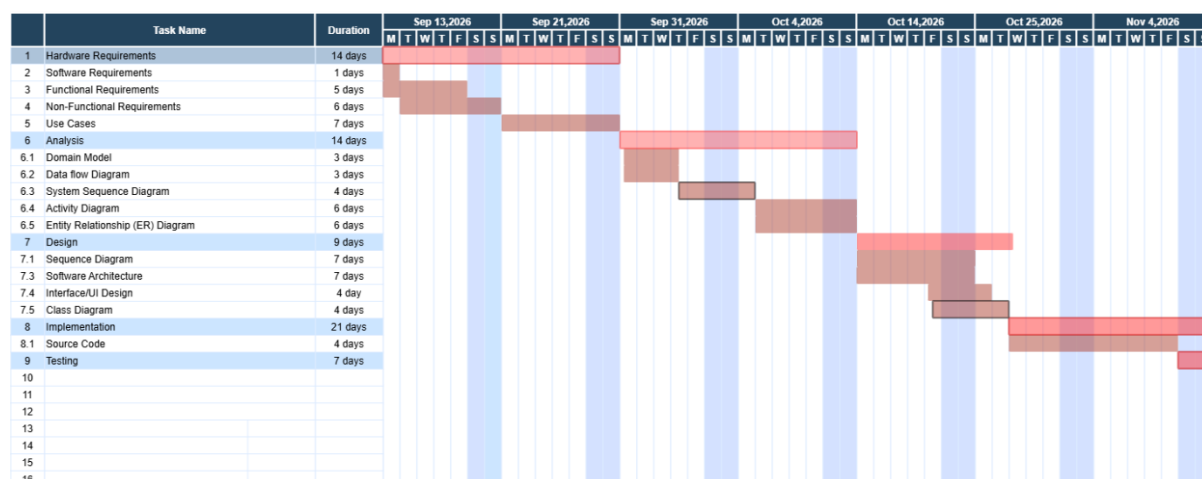
The project follows an Incremental Development Model: the system is built step by step, with each increment adding functionality and improving the previous one. This model was chosen because requirements are mostly clear but small changes are expected, it allows early testing of the database and API before the full system exists, and each increment produces a working version that can be reviewed.

Increment	Planned functionality	Indicative phases
Increment 1	Database design and basic data-entry system.	Phases 2–3
Increment 2	API development and user authentication.	Phases 3–4
Increment 3	Search, filtering, export and final testing.	Phases 4–5

5.2 Project Schedule

Phase	Activity	Duration	Weeks
Phase 1	Requirements gathering and analysis	2 weeks	1–2
Phase 2	System design (DB schema, API design)	2 weeks	3–4
Phase 3	Backend development and database setup	3 weeks	5–7
Phase 4	API development and testing	2 weeks	8–9
Phase 5	Documentation and final review	1 week	10

Timeline overview (shaded cells show the weeks in which each phase is active):



5.3 Work Activities

Phase	Work activities
Phase 1	Collect and analyze requirements; define scope, user roles, use cases and the SRS.
Phase 2	Design the PostgreSQL schema and ER diagram; design REST endpoints, JWT authentication and the web interface pages.
Phase 3	Set up PostgreSQL; build the Node.js/Express backend, data-entry features (animals, DNA, health) and role-based access.
Phase 4	Build and test API endpoints (movements, CSV upload, search, export); run unit and integration tests in Postman.
Phase 5	Run system and acceptance tests; finalize documentation; conduct final review before submission.

5.4 Milestones

ID	Milestone	Target
M1	Requirements baselined (SRS complete)	End of week 2
M2	Database schema, API and interface design complete	End of week 4
M3	Backend and database operational; basic data entry working	End of week 7
M4	API complete and unit/integration tested	End of week 9
M5	System and acceptance testing passed; final documentation submitted	End of week 10

5.5 Resource Allocation

The SRS defines the technology stack and minimum hardware. Technical resources are:

Resource	Specification / purpose
Development machine	Intel Core i3 (or equivalent) 2.0 GHz; 4 GB RAM minimum (8 GB recommended for development).
Database storage	At least 50 GB free disk space for the database.
Deployment server	2 CPU cores, 4 GB RAM, 100 GB SSD cloud instance.
Network	Stable internet connection for portal and API access.
Software	Windows 10 / Ubuntu 20.04, Node.js v18+, PostgreSQL 14+, Postman, VS Code, Git, Chrome/Firefox.

5.6 Team and Responsibilities

Team member	Role
Muhammad Abbas	Project team member
Muhammad Ehtisham	Project team member and Leader
Fahad Naeem Khan	Project team member

Work is shared across the team. Each member is assigned specific issues/tasks on the GitHub repository through GitHub Projects, so progress and ownership are visible and every task is completed on time.

5.7 Project Management and Communication

- **GitHub Projects:** task board with three columns — To Do (tasks not started), In Progress (tasks being worked on) and Done (completed tasks).
- **Git / GitHub:** version control and issue tracking.
- **WhatsApp and Discord:** day-to-day team communication.
- **Google Docs:** shared documentation.
- **Weekly meetings:** every Saturday to review progress against the schedule.

6. Supporting Process Plans

6.1 Quality / Validation Plan

Data quality is supported through mandatory fields, validation at submission, row-by-row CSV validation, coordinate integrity checks, unique identifiers, and timestamps with audit logs recording who created or modified each record. The API returns standard HTTP status codes (for example 400, 401, 403, 404, 409) so clients can handle errors consistently.

6.2 Testing Plan

Black Box Testing is the main approach. Tests are designed from input values and expected outputs only, which suits the API endpoints, validates functional requirements without reading code, and reflects how real users and external applications will use WIS.

Test level	Approach	When
Unit testing	Test each API endpoint individually using Postman.	After each endpoint, during development
Integration testing	Test all API endpoints together.	After the full API build (after Increment 2)
System testing	Test the complete flow from login through data export.	Final week
Acceptance testing	Verify the final system against the requirements.	Before submission

Test tools: Postman for manual API testing and browser DevTools for the web interface.

6.3 Test Cases

ID	Feature	Input	Expected output
TC-01	User login	Valid email and password	JWT token returned, status 200
TC-02	User login (wrong)	Valid email, wrong password	Status 401 Unauthorized
TC-03	Add animal	Valid animal data fields	Record saved, new ID returned
TC-04	Add animal (missing)	Species name missing	Status 400, field error message

ID	Feature	Input	Expected output
TC-05	Get animal by ID	Valid animal ID in URL	Full animal record in JSON
TC-06	Get animal (wrong ID)	ID that does not exist	Status 404 Not Found
TC-07	Search by species	species=Snow Leopard	List of matching animals
TC-08	CSV upload (valid)	Correctly formatted CSV	All rows saved successfully
TC-09	CSV upload (bad)	CSV with missing coordinates	Rows skipped, error report shown
TC-10	API without token	GET request without Auth header	Status 401 Unauthorized
TC-11	Export data	Export search results	JSON or CSV file downloaded
TC-12	Admin creates user	New user details with role	User account created successfully

6.4 Configuration / Version Control Plan

All source code, database scripts and documentation are kept in a shared Git/GitHub repository. Work is tracked as GitHub issues on the GitHub Projects board (To Do, In Progress, Done), and each member works on assigned issues. Shared documents are maintained in Google Docs, and the team reviews changes in the weekly Saturday meeting.

6.5 Acceptance Criteria

- Users can log in and receive an access token.
- An authorized user can add an animal and retrieve it through the API.
- Movement CSV data can be uploaded and saved when valid.
- Search filters return the appropriate animals by species or geographic location.
- The API returns appropriate HTTP errors (401, 400, 404).
- An administrator can create a user and assign a role.

7. Risk Management

The risk register below is a planning addition derived from the risks implied by the SRS. The team should review it at the weekly meeting and update it as development progresses.

ID	Risk	Potential impact	Priority	Mitigation / response
R1	Network dependency	Users cannot reliably access the portal or API.	High	Require stable network; return clear errors and standard HTTP responses.
R2	Data quality	Invalid or inconsistent wildlife data enters the database.	High	Mandatory fields, validation, CSV row checks, unique IDs and audit logs.
R3	Unauthorized access	Sensitive wildlife data exposed to an improper user.	High	JWT authentication, role-based access, bcrypt hashing, authorization checks on every endpoint.

ID	Risk	Potential impact	Priority	Mitigation / response
R4	Large data volume	Queries slow down as records grow toward 1 million.	Medium	Database indexing, query optimization, pagination; test with large data sets.
R5	External tracking dependency	GPS data quality or availability depends on external devices.	Medium	Treat WIS as a repository only; validate incoming coordinates.
R6	Scope expansion	Real-time maps, mobile apps, AI/ML or international integrations delay delivery.	Medium	Keep excluded features out of v1.0; log them as future phases.
R7	Schedule slippage	Ten-week schedule is short for a three-member team.	Medium	Track progress weekly on GitHub Projects; deliver by increment so a working version exists early.

Risk Monitoring

Risks are reviewed in the weekly Saturday meeting. If a risk occurs, the team records it on the GitHub board, assigns an owner and applies the mitigation. Priority levels (High, Medium, Low) reflect combined likelihood and impact and may be revised as the project progresses.

8. Key System Interface and API Requirements

Method	Endpoint	Purpose
GET	/api/animals	Get animals with filters.
GET	/api/animals/:id	Get a single animal by ID.
POST	/api/animals	Add an animal record.
PUT	/api/animals/:id	Update an animal record.
DELETE	/api/animals/:id	Delete an animal record.
GET	/api/animals/:id/movements	Get movement history for an animal.
POST	/api/movements/upload	Bulk upload CSV movement data.
GET	/api/animals/:id/health	Get health records for an animal.
GET	/api/animals/:id/dna	Get DNA records for an animal.
POST	/api/auth/login	Authenticate a user and return a JWT.
GET	/api/species	Get the list of all species.

Web Interface Pages

Login page, dashboard, animal list, add-animal form, movement records page (with CSV upload), API documentation page and admin-only user management. The interface uses a minimal blue-and-white design; errors are shown in red and success messages in green.

9. Implementation Status

According to the SRS, the implementation stage had not yet begun. The system design, database model and API structure have been designed, and coding is planned as future work following the designs in the SRS (Section 4) and the schedule in this plan.

10. Traceability to the Plan Outline

Plan outline item	Where addressed in this Project Plan
Front matter	Cover page, Document Control, Revision History
Project overview	Section 1
References	Section 2
Definitions	Section 3
Project context	Section 4
Project planning	Section 5
Supporting process plans	Section 6
Risk management	Section 7